# AI-101L Comprehensive Master Quiz Preparation Guide

## Conceptual Deep Dive & Exam Cheat Sheet (Labs 01 – 05)

**Course:** Introduction to Artificial Intelligence (AI-101L)

**Instructor:** Ali Hassan Sherazi

**Student Portfolio:** [dropgaming786-collab/ai-lab](https://github.com/dropgaming786-collab/ai-lab)

**Target:** Theoretical Mastery & Conceptual Quiz Preparation

---

## Quick Navigation


1. [General Exam Strategy & Core Principles](#1-general-exam-strategy--core-principles)
2. [Lab 01: Environment, Anaconda, Git & Exploratory Data Inspection](#2-lab-01-environment-anaconda-git--exploratory-data-inspection)
3. [Lab 02: Polite Web Scraping, NLP Feature Extraction & EDA](#3-lab-02-polite-web-scraping-nlp-feature-extraction--eda)
4. [Lab 03: Multimodal Feature Engineering (Text & Images)](#4-lab-03-multimodal-feature-engineering-text--images)
5. [Lab 04: Scikit-Learn Fundamentals & Traditional Machine Learning](#5-lab-04-scikit-learn-fundamentals--traditional-machine-learning)
6. [Lab 05: Classification as an ML Task (Titanic Survival System)](#6-lab-05-classification-as-an-ml-task-titanic-survival-system)
7. [High-Probability Conceptual Quiz Questions & Model Answers](#7-high-probability-conceptual-quiz-questions--model-answers)
8. [Formula & Metric Reference Cheat Sheet](#8-formula--metric-reference-cheat-sheet)


---

## 1. General Exam Strategy & Core Principles

In AI-101L quizzes, instructors test **why** design choices are made, **what happens if assumptions fail**, and **mathematical / architectural trade-offs**, rather than rote syntax. Keep these five overarching principles in mind:

1. **The Baseline Principle:** Always identify the Zero-Rule (majority-class) baseline before evaluating a model. If class imbalance is 62/38, a model predicting the majority class scores 62% without learning anything.
2. **The Leakage Axiom:** Preprocessing parameters (mean, variance, mode, vocabulary, IDF weights) must be derived **only from training partitions**. Test/validation partitions must only be transformed.
3. **The Bias-Variance Tradeoff:** Simple linear models have high bias and low variance; complex unpruned non-linear models (Decision Trees) have low bias and high variance. Small datasets ($N \leq 100$) amplify variance catastrophically.
4. **The Scale Invariance Nuance:** Unregularized OLS linear regression is invariant to linear feature scaling ($R^2$ is unchanged). Distance-based (k-NN, SVM), regularized (Ridge, Lasso), and gradient-descent algorithms are **not** invariant.
5. **The Representation Principle:** Raw data (HTML, text strings, RGB pixels) cannot enter mathematical loss functions; they must be mapped to vector spaces ($R^d$) preserving task-relevant semantic geometry.

---

# 2. Lab 01: Environment, Anaconda, Git & Exploratory Data Inspection

## 2.1 Why Isolated Virtual Environments (Conda)?

- **Dependency Conflicts (Dependency Hell):** Different projects require incompatible versions of shared C-libraries or packages (e.g., PyTorch needing CUDA 11 vs CUDA 12, or conflicting NumPy ABI versions).
- **Reproducibility:** A pinned `requirements.txt` (`pip freeze`) or `environment.yml` guarantees that an experiment runs identically across developer machines, CI/CD runners, and production servers.
- **Non-Destructive Experimentation:** If an environment becomes corrupted, it can be deleted (`conda env remove -n ai-lab`) without affecting system Python or other projects.

## 2.2 Why Inspect Before Modifying Data?

- **Silent Corruption:** Applying imputation or type casting blindly (e.g., filling nulls with mean) alters the underlying distribution before you even understand if the missingness is **MCAR** (Missing Completely at Random), **MAR** (Missing at Random), or **MNAR** (Missing Not at Random).
- **Outlier Masking:** Cleaning without inspecting masks anomalies that may represent critical signal (fraud detection) or sensor failures (faulty readings).

## 2.3 Key Statistical Metrics

- **Skewness:** Measure of distribution asymmetry about the mean.
- *Positive (Right) Skew:* Tail extends to the right; {Mode} < {Median} < {Mean}. Common in financial data (salaries, housing prices). Mean is pulled up by extreme outliers; **Median is the robust measure of central tendency**.
- *Negative (Left) Skew:* Tail extends to the left; {Mean} < {Median} < {Mode}.
- **Kurtosis:** Measure of the "tailedness" and peakedness relative to a normal distribution. High kurtosis (leptokurtic) indicates heavy tails and extreme outlier susceptibility.
- **Coefficient of Variation (CV):**

CV = {σ}{μ}

- Standard deviation normalized by the mean.
- **Why it matters:** Unlike standard deviation σ (which depends on physical units), CV is dimensionless. A feature with CV = 2.5 has massive relative dispersion compared to one with CV = 0.1. In unscaled models, features with artificially large variance dominate loss functions.

## 2.4 Git Architecture & Version Control Mechanics

- **Three-Tree Architecture:**

1. **Working Directory:** Actual files on disk where editing occurs.
2. **Staging Area (Index):** Snapshot of changes queued for the next commit (`git add`). Decouples editing from committing.
3. **Repository (Commit History):** Cryptographically hashed (SHA-1) permanent directed acyclic graph (DAG) of project states (`git commit`).

- **Branching & Merging:**
- **Fast-Forward Merge:** Target branch has no new commits since the feature branch diverged; Git simply moves the pointer forward.
- **3-Way Merge:** Both branches have independent commits; Git finds the common ancestor and creates a new merge commit.
- **Merge Conflicts:** Occur when competing changes modify the exact same lines in a file. Git halts and requires human resolution.

# 3. Lab 02: Polite Web Scraping, NLP Feature Extraction & EDA

## 3.1 NumPy Vectorization vs Native Python Loops

- **The CPU Reality:** Native Python loops execute via the CPython interpreter bytecode evaluation loop. Every iteration incurs:

1. Dynamic type checking and pointer dereferencing.
2. Memory fragmentation (Python integers/floats are heap-allocated objects with header overhead).
3. Lack of compiler loop unrolling.

- **Vectorized Execution:** NumPy arrays store contiguous blocks of homogeneous C data types (`int32`, `float64`). Vectorized operations leverage:
- Low-level SIMD (Single Instruction, Multiple Data) CPU vector registers.
- Hardware cache locality (L1/L2 prefetching).
- Direct execution in optimized BLAS/LAPACK C and Fortran libraries.
- **Measured Speedup:** Typically **50× to 500× faster** than native `for` loops.

## 3.2 Web Scraping Protocol & Politeness

- **Polite Scraping Checklist:**

1. `robots.txt` compliance: Check which paths the site owner disallows.
2. `User-Agent` Header: Identifies your scraper cleanly to the server administrator.
3. Rate Limiting (`time.sleep`): Prevents Denial-of-Service (DoS) and server throttling (HTTP 429 Too Many Requests).
4. Response Caching: Save raw scraped text (`articles_raw.csv`) so re-running EDA doesn't re-query the web server.

- **DOM vs RegEx for HTML Parsing:**
- Regular expressions cannot parse arbitrary HTML because HTML is a Context-Free Grammar with nested, recursive tags, while RegEx is restricted to Regular Languages. Beautiful Soup constructs a Document Object Model (DOM) parse tree that handles malformed markup gracefully.

## 3.3 NLP Feature Engineering

- **spaCy Pipeline Architecture:**
- Raw text → Tokenizer → Tagger (POS) → Parser (Dependencies) → NER (Named Entities).
- **Lemmatization vs Stemming:**
- *Stemming (e.g. Porter):* Crude heuristic stripping of suffixes (`studies` → `studi`). Fast, but produces non-words.
- *Lemmatization (spaCy):* Uses vocabulary and morphological vocabulary lookup to return the canonical dictionary base form (`studies` → `study`).
- **Noun Density:** Ratio of nouns to total tokens ({{num_nouns}}{{num_tokens}}). High noun density indicates information-dense technical or academic text; low density indicates conversational narrative.
- **Sentiment Analysis (TextBlob):**
- **Polarity** $\in$ **[-1.0, +1.0]:** Degree of positive vs negative emotion (lexicon-derived word scores adjusted for negation and intensifiers).
- **Subjectivity** $\in$ **[0.0, 1.0]:** Objective factual statements (0.0) vs subjective opinions/beliefs (1.0).

## 3.4 TF-IDF (Term Frequency - Inverse Document Frequency)

{TF-IDF}(t, d, D) = {TF}(t, d) × {IDF}(t, D)

{IDF}(t, D) = ln({1 + |D|}{1 + {DF}(t)}) + 1

- **Intuition:**

- {TF}: How often word t appears in document d (local relevance).
- {IDF}: How rare word t is across all documents in corpus D (global discriminative power).
- Stop words ("the", "is", "at") appear in every document ({DF} ≈ |D|), driving {IDF} → 0. Rare domain terms ("algorithm", "transformer") have high {IDF} and dominate the feature vector.
- **KDE Plot Caveat:**
- Kernel Density Estimation (KDE) places a Gaussian kernel over every data point and sums them. For tiny sample sizes (N=5), KDE is **not** a valid population density estimate; bandwidth smoothing produces artificial probability mass in impossible regions (e.g., negative token counts).

---

# 4. Lab 03: Multimodal Feature Engineering (Text & Images)

## 4.1 Bag-of-Words (BoW) & N-Grams

- **BoW Definition:** Disregards grammar and word order; represents documents as a vector of vocabulary word frequencies.
- **What BoW Throws Away:**

1. *Word Order:* "dog bites man" and "man bites dog" yield identical BoW vectors.
2. *Context & Negation:* "not happy" and "happy" share the word "happy".
3. *Semantic Similarity:* Synonyms ("car" and "automobile") are treated as orthogonal, unrelated features.

- **N-Grams:** Sequences of N consecutive tokens.
- *Unigram (N=1):* Individual words ("machine", "learning").
- *Bigram (N=2):* Word pairs ("machine learning", "not guilty"). Captures immediate context and negation.
- *The N-Gram Trade-off:* Higher N captures richer context, but the feature space grows exponentially (|V|^N), leading to severe sparsity and vocabulary explosion.

## 4.2 Image Feature Preprocessing Pipeline

Raw raster images consist of discrete grid pixels with spatial coordinates (x, y) and colour channels (R, G, B):

1. **Grayscale Conversion:** Reduces dimensionality by 3× (3 channels → 1 intensity channel) using the standard human perceptual luminosity formula:

Y = 0.299\,R + 0.587\,G + 0.114\,B

(Human visual receptors are most sensitive to green, moderately to red, and least to blue).

2. **Standardized Resizing:** Machine learning classifiers require fixed-length feature vectors {x} ∈ R^d. Interpolation algorithms (Bilinear, Bicubic) downsample or upsample diverse camera resolutions to a standard matrix (e.g., 64 × 64).
3. **Flattening:** Unrolls the 2D matrix (H, W) into a 1D vector {x} ∈ R^{H × W}. For 64 × 64, {x} ∈ R^{4096}.

- *Limitation of Flattening:* Destroys 2D spatial adjacency and translational invariance (why CNNs later replace flattened MLPs).

## 4.3 The Curse of Dimensionality (p >> n)

In Lab 03, the image dataset contained N = 20 samples and d = 4,096 features (ratio > 200:1).

- **Geometric Consequence:** As dimensionality increases, the volume of the space grows exponentially (V ∝ r^d). Data points become isolated in the outer corners of the hypercube.
- **Distance Concentration:** The ratio of distances between the nearest and farthest neighbor converges to 1:

\lim_{d → ∈fty} {{dist}_{max} - {dist}_{min}}{{dist}_{min}} → 0

In 4096-D space, Euclidean distance ceases to be discriminative; all points appear equidistant.

- **Overfitting:** Any unconstrained linear classifier can trivially find a separating hyperplane that perfectly classifies noise without learning actual visual features.
- **Remedies:** Dimensionality reduction (PCA), feature selection, L1 regularization (Lasso), convolutional feature extractors, or collecting vastly more data (N >> d).

---

# 5. Lab 04: Scikit-Learn Fundamentals & Traditional Machine Learning

## 5.1 The Scikit-Learn Unified Estimator API Contract

All algorithms in Scikit-learn adhere to a consistent object-oriented interface:

- `Estimator`: Learns from data via `.fit(X, y)`. Modifies internal state; trailing underscore attributes (`coef_`, `classes_`) denote fitted parameters.
- `Transformer`: Filters or modifies data via `.transform(X)`. Implements atomic `.fit_transform(X)` for training efficiency.
- `Predictor`: Makes predictions on unseen data via `.predict(X)` or `.predict_proba(X)`.

## 5.2 Ordinary Least Squares (OLS) Regression

- **Objective:** Minimizes the Residual Sum of Squares (RSS):

min_{{w}} Σ_{i=1}^n ( y_i - {w}^T {x}_i - b )^2

- **Closed-form Normal Equation:** {w}^* = ({X}^T {X})^{-1} {X}^T {y}.
- **Coefficient of Determination (R^2):**

R^2 = 1 - {{SS}_{{res}}}{{SS}_{{tot}}} = 1 - {Σ (y_i - {y}_i)^2}{Σ (y_i - {y})^2}

Proportion of variance in target y explained by features {X}.

## 5.3 Standardisation Invariance in Plain OLS

- **The Puzzle:** In Lab 04, training Linear Regression on raw features gave R^2 = 0.575788. Training on `StandardScaler`-transformed features gave identical R^2 = 0.575788.
- **Mathematical Rationale:**

Linear scaling is an affine transformation: {X}_{{scaled}} = ({X} - μ) {S}^{-1}.

Because the column space (span) of {X} is invariant under non-singular linear transformations:

{span}({X}_{{scaled}}) = {span}({X})

OLS finds the orthogonal projection of {y} onto this column space. The fitted values {{y}}, residuals {e}, and R^2 remain mathematically identical; only the coefficients rescale: {w}_{{raw}} = {S}^{-1} {w}_{{scaled}}.

- **Where Standardization DOES Alter Performance:**

1. *Regularized Models (Ridge, Lasso, ElasticNet):* Penalty terms λ \|{w}\|_2^2 or λ \|{w}\|_1 penalize large weights equally. Unscaled features with small raw numbers require huge weights and get unfairly penalized.
2. *Distance-Based Models (k-NN, SVM, k-Means):* Euclidean distance √{Σ (x_j - z_j)^2} is completely dominated by large-scale features.
3. *Gradient Descent Algorithms (Neural Networks, LogisticRegression with solvers):* Spherical contour lines yield straight descent paths; elongated ellipses cause slow, oscillating zig-zag convergence.

## 5.4 Raw vs Standardized Regression Coefficients

- **Why Raw β Deceives:** Raw weights depend inversely on feature units:

β_j = {Δ y}{Δ x_j}

If x_j is measured in dollars (large numbers), β_j is tiny. If x_j is measured in fractions of bedrooms, β_j is huge.

- **Standardized Coefficients (β^*):**

β_j^* = β_j × {σ(x_j)}{σ(y)}

Reflects the change in y (in standard deviations) per one-standard-deviation change in x_j. Enables true ranking of feature importance.

### 5.5 Classification & The Confusion Matrix

- **Iris Dataset Insight:**
- Iris Setosa is linearly separable from the other two species along petal dimensions.
- **Versicolor and Virginica** have overlapping petal length and width distributions, causing mutual misclassifications in tighter linear splits.
- **Text Sentiment Pipeline:**
- `Pipeline([('tfidf', TfidfVectorizer()), ('clf', LogisticRegression())])`.
- Encapsulates vocabulary building, IDF scoring, and classifier training into one serialized `.pkl` or `.joblib` file. Inference can be called on raw text strings (`pipe.predict(["This is great!"])`) with zero manual tokenization.

---

## 6. Lab 05: Classification as an ML Task (Titanic Survival System)

### 6.1 The 6-Stage Machine Learning Cycle

{Load \& Explore} → {Preprocess} → {Split \& Scale} → {Train \& Compare} → {Tune} → {Persist}

- **Hinge of the Course:** Everything prior produced *features*; everything after consumes the *persisted model* (Lab 06 wraps it in FastAPI; Labs 11-12 build a web UI; Labs 14-16 add logging, drift monitoring, and feedback retraining).

### 6.2 The No-Information Baseline Rule

- In the Titanic dataset, 62\% of passengers died (y=0) and 38\% survived (y=1).
- A naive baseline classifier that always predicts the majority class ("died") achieves **62\% accuracy**.
- **Golden Rule:** Any machine learning model that achieves ≤ 62\% accuracy has learned **no useful signal**. Accuracy alone without the baseline context is meaningless.

### 6.3 Data Leakage: The Silent Killer of Machine Learning

- **What is Data Leakage?** When information from outside the training partition (specifically test or validation holdout sets) inadvertently contaminates the model training process.
- **The Manual Preprocessing Flaw:**

If you impute missing values (mean/mode), scale features (`StandardScaler`), or compute target encodings over the **entire dataset before train/test splitting**, the training partition learns properties of the test distribution.

- **Consequences:** The model achieves artificially inflated, overly optimistic test scores during development, but suffers catastrophic performance drops when deployed to live real-world data.
- **The Solution: Scikit-Learn Pipeline & ColumnTransformer:**

`Pipeline` guarantees that when performing 5-fold cross-validation:

{Fold } k: {Transformers call } tt{.fit_transform()} { ONLY on train fold } D_{{train}}^{(k)}

{Transformers call } tt{.transform()} { on holdout fold } D_{{val}}^{(k)}

Zero distribution statistics from validation folds bleed into the training pipeline.

### 6.4 Model Comparison: Strengths, Weaknesses & Trade-Offs

| Model Family | Core Mechanism | Strengths | Vulnerabilities / Gotchas | Small Sample (N=100) Behavior |
|---|---|---|---|---|
| **Logistic Regression** | Linear combination through Sigmoid σ({w}^T {x} + b) | Calibrated probabilities, fast inference (<1{ms}), convex loss (no local minima) | Cannot model non-linear interactions without explicit polynomial features | **Best generalization:** Low variance, tightly bounded CV score |
| **Decision Tree (CART)** | Recursive binary feature space partitioning via Gini/Entropy | Highly interpretable, handles mixed feature types, no scaling required | Unstable (high variance), greedily overfits, sensitive to tiny data perturbations | **Overfits easily:** Single sample shift radically alters tree splits |
| **Random Forest** | Bagging (bootstrap aggregation) + random feature subspace splits | Reduces tree variance, robust to outliers, computes feature importances | Black-box, slower inference, unpruned trees can still overfit small datasets | High variance unless `max_depth` is constrained (tuned: 73\%) |
| **K-Nearest Neighbors** | Non-parametric, distance-weighted majority vote of k nearest instances | Zero training time (lazy learner), captures complex local manifolds | O(N) inference complexity, suffers from curse of dimensionality, sensitive to unscaled features | Low stability on integer categorical features (63\%) |
| **Support Vector Machine (SVC)** | Finds maximum-margin hyperplane; RBF kernel maps to ∈fty-D Hilbert space | Robust to high dimensions, global margin regularizer | Slow O(N^3) training on large data, uncalibrated probabilities require Platt scaling | **Tightest variance band:** Strong regularization on small samples |

### 6.5 Evaluation Metrics Under Class Imbalance

When positive class (survived) is the minority:

- **Accuracy:** {TP + TN}{TP + TN + FP + FN}. Misleading under imbalance (predicting all 0 gives 62%).
- **Precision:** {TP}{TP + FP}. "Of all passengers we predicted would survive, how many actually did?" (Cost of False Alarm).
- **Recall (Sensitivity):** {TP}{TP + FN}. "Of all actual survivors, how many did we find?" (Cost of False Negative).
- **F1-Score:** Harmonic mean of Precision and Recall:

F_1 = 2 × {{Precision} × {Recall}}{{Precision} + {Recall}}

Forces a balance; if either metric approaches zero, F_1 → 0.

- **ROC Curve & AUC:**
- Plots True Positive Rate (Sensitivity) vs False Positive Rate (1 - {Specificity}) across all classification thresholds t ∈ [0, 1].
- **AUC = 0.5:** Random guessing (diagonal line).
- **AUC = 1.0:** Perfect discrimination.
- **Threshold Independence:** AUC measures ranking ability independent of the arbitrary 0.5 decision cutoff.

---

## 7. High-Probability Conceptual Quiz Questions & Model Answers

### Question 1: Data Preprocessing & Leakage

**Q:** A data scientist applies `StandardScaler().fit_transform(df)` to the complete dataset, splits the result using `train_test_split`, and trains a model that achieves 94% test accuracy. In production, accuracy plummets to 68%. Explain the exact methodological mistake and why it caused this failure.

**Model Answer:**

The data scientist committed **Data Leakage**. By calling `fit_transform` on the entire dataset prior to splitting, the global mean μ and standard deviation σ of the test set were computed and incorporated into the feature values seen by the training set. The model memorized statistical boundaries influenced by the test set rather than learning true generalization. In production, real-world data has unseen distribution shifts and lacks test set contamination, exposing the model's severe overfitting. The correct approach is to split first, call `scaler.fit_transform(X_train)`, and call `scaler.transform(X_test)`.

---

## Question 2: OLS Linear Regression & Feature Scaling

**Q:** You multiply all values of an independent feature x_1 in a dataset by a constant c = 1000 (e.g. converting grams to milligrams). How does this affect: (a) the OLS regression coefficient β_1, (b) the prediction {y}, and (c) the R^2 score? Would your answer change if using Ridge Regression?

**Model Answer:**

- **(a) β_1:** Scaled by {1}{c} = {1}{1000}. Since y = ... + β_1 x_1, multiplying x_1 by 1000 forces the optimal weight to divide by 1000 to maintain the same fitted values.
- **(b) {y}:** Unchanged. The linear column space is invariant under non-zero scalar multiplication.
- **(c) R^2:** Unchanged. Residual sum of squares and total sum of squares are identical.
- **Ridge Regression: YES, the answer changes completely!** Ridge minimizes {RSS} + λ Σ β_j^2. If x_1 is in milligrams (values 1000x larger), its raw coefficient is tiny (1/1000). The penalty λ β_1^2 becomes negligible compared to other features, effectively exempting x_1 from regularization while heavily shrinking other features.

---

## Question 3: Tree Models vs Linear Models on Feature Engineering

**Q:** In Lab 05, adding engineered features {FamilySize} = {Sibling} + 1 and {IsAlone} = ({FamilySize} == 1) produced zero accuracy delta on test data for Decision Trees and Random Forests. Why are tree algorithms naturally indifferent to this feature engineering, and why might linear models benefit?

**Model Answer:**

Decision Trees partition feature space using orthogonal axis-aligned splits on individual features (e.g., {Sibling} ≤ 0). Since {Sibling}=0 corresponds directly to solo travelers, a tree can isolate solo passengers with a single threshold without needing an explicit linear sum ({Sibling} + 1). The monotonic transformation preserves feature ranking. Conversely, standard linear models assume additive effects: y = w_1 · {Sibling}. A linear model cannot naturally model step-function thresholds (e.g. solo passengers having dramatically lower survival while families of 2-4 survive well, but large families of 6+ perish) without explicit indicator features like {IsAlone} or interaction terms.

---

## Question 4: Bag-of-Words vs N-Grams

**Q:** Consider the two movie reviews: Review A: "The plot was not good, acting was terrible." Review B: "The plot was good, acting was not terrible." Explain why a Unigram Bag-of-Words representation fails to distinguish their sentiments, and how Bigrams solve this.

**Model Answer:**

A Unigram BoW counts individual word frequencies. Both Review A and Review B contain identical word sets: `{"the": 1, "plot": 1, "was": 2, "not": 1, "good": 1, "acting": 1, "terrible": 1}`. Their resulting BoW vectors are mathematically identical, forcing any classifier to assign identical predictions despite opposite sentiments. Bigrams (N=2) capture pairwise sequential adjacency: Review A contains `("not", "good")` and `("was", "terrible")`, whereas Review B contains `("was", "good")` and `("not", "terrible")`. The negation tokens are bound to their respective adjectives, allowing the classifier to separate the vectors.

---

### Question 5: Evaluation Metric Selection

**Q:** An AI startup develops a computer vision classifier for detecting malignant skin cancer from dermatoscope images (N=10,000, where 9,900 are benign and 100 are malignant). The model achieves 99.0% accuracy. Why is this metric dangerous, and which metric should the lead engineer demand?

**Model Answer:**

The 99.0% accuracy is an illusion driven by extreme class imbalance (99:1). A trivial dummy model that predicts "benign" for every single patient scores 99.0% accuracy while failing to detect a single cancer patient (100\% mortality). In medical diagnostic screening, the cost of a **False Negative** (telling a sick patient they are healthy) is fatal, whereas a **False Positive** merely results in an additional non-invasive biopsy. The lead engineer must demand **High Recall (Sensitivity)** ({TP}{TP + FN}) and evaluate the model using **ROC-AUC** or **Precision-Recall AUC (PR-AUC)**.

---

### Question 6: Curse of Dimensionality in Image ML

**Q:** In Lab 03, we flattened 64 × 64 grayscale images into 4,096-dimensional vectors for a dataset of 20 images. Explain what happens to Euclidean distances in this space and why a k-NN classifier fails.

**Model Answer:**

In a 4,096-dimensional space with only 20 samples (p >> n), the volume of the space is astronomically large compared to the sample density. In high dimensions, pairwise Euclidean distances √{Σ_{j=1}^{4096} (x_{ij} - x_{kj})^2} concentrate around a common mean: the relative difference between the distance to the nearest neighbor and the distance to the farthest neighbor approaches zero. Consequently, all images appear almost equidistant from one another. A k-NN classifier relies on distance metrics to identify semantic neighbors; when distances lose contrast, nearest-neighbor selection becomes equivalent to random selection.

---

## 8. Formula & Metric Reference Cheat Sheet

| Concept / Metric | Mathematical Formula | Key Intuition / Interpretation |
|---|---|---|
| **Coefficient of Variation** | CV = {σ}{μ} | Relative dispersion independent of measurement units |
| **Inverse Document Frequency** | {IDF}(t) = ln({1 + N}{1 + {DF}(t)}) + 1 | Penalizes ubiquitous words; boosts rare discriminative terms |
| **Luminosity Grayscale** | Y = 0.299R + 0.587G + 0.114B | Perceptual weighting aligned with human eye cone sensitivity |
| **OLS Cost Function** | J({w}) = {1}{2n} Σ_{i=1}^n (y_i - {y}_i)^2 | Residual Sum of Squares (Mean Squared Error) |
| **R^2 Score** | R^2 = 1 - {{SS}_{{res}}}{{SS}_{{tot}}} | Proportion of target variance explained by regression line |
| **Sigmoid Function** | σ(z) = {1}{1 + e^{-z}} | Maps real-valued score (-∈fty, ∈fty) to probability (0, 1) |

| | | |
|---|---|---|
| **Zero-Rule Baseline** | {Acc}_{{base}} = max({N_0}{N}, {N_1}{N}) | Performance obtained by blindly predicting majority class |
| **Precision** | {TP}{TP + FP} | Positive predictive value (Penalty for False Alarms) |
| **Recall (Sensitivity)** | {TP}{TP + FN} | True positive detection rate (Penalty for Misses) |
| **Specificity** | {TN}{TN + FP} | True negative rate (1 - {FPR}) |
| **F1-Score** | 2 · {{Precision} · {Recall}}{{Precision} + {Recall}} | Harmonic mean balancing Precision and Recall |
| **Decision Tree Gini Impurity** | I_G(p) = 1 - Σ_{k=1}^K p_k^2 | Probability of misclassifying a randomly chosen element |

---

*This study guide was compiled from the code, logs, and technical reports of Labs 01 through 05 in the `ai-lab` portfolio. Master these concepts for your quiz.*